

Local HTTPS Setup for Django Using mkcert (Development & Testing)

Purpose

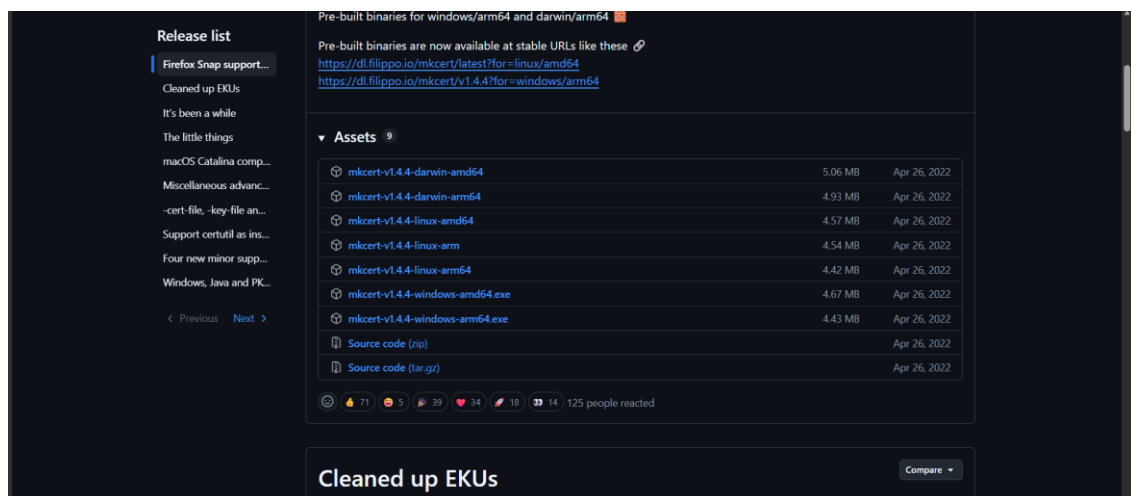
This document explains how to enable **HTTPS** for a Django project using **mkcert**. This setup is useful for **development and testing only**, especially when testing:

- Browser Push Notifications
- Firebase Cloud Messaging (FCM)
- Service Workers
- Notification API
- Other browser features that require HTTPS

Note: This setup is **not for production**.

Step 1: Download mkcert

1. Visit the mkcert GitHub Releases page.
2. Download the latest **Windows AMD64** version.



3. Rename the downloaded file to:
mkcert.exe

4. Move it to:
C:\Program Files\

Your file should now be located at:

C:\Program Files\mkcert.exe

Step 2: Install the Local Certificate Authority (CA)

Open **Command Prompt** as **Administrator**.

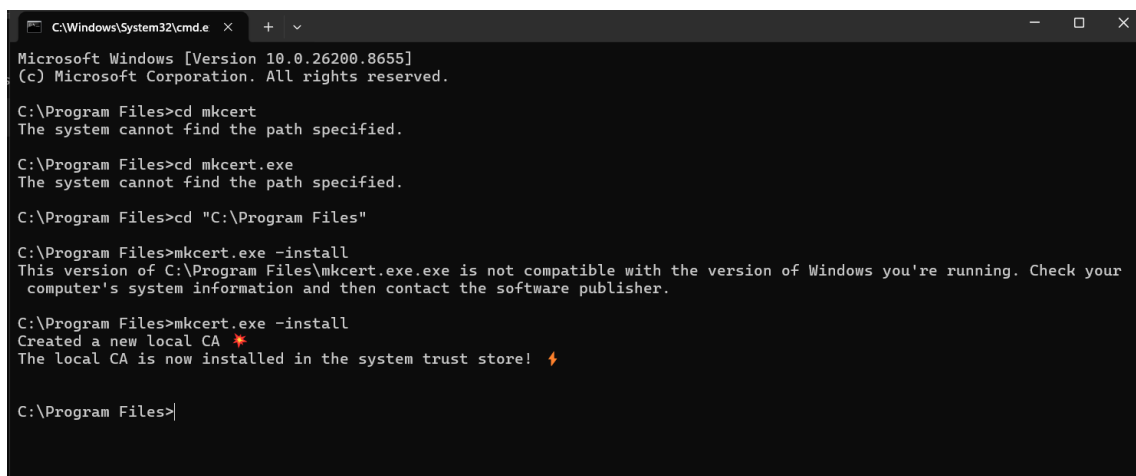
Run:

```
"C:\Program Files\mkcert.exe" -install
```

If successful, you will see:

Created a new local CA

The local CA is now installed in the system trust store.

A screenshot of a Windows Command Prompt window. The title bar shows 'C:\Windows\System32\cmd.exe'. The window content shows the following commands and output:

```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Program Files>cd mkcert
The system cannot find the path specified.

C:\Program Files>cd mkcert.exe
The system cannot find the path specified.

C:\Program Files>cd "C:\Program Files"

C:\Program Files>mkcert.exe -install
This version of C:\Program Files\mkcert.exe.exe is not compatible with the version of Windows you're running. Check your
computer's system information and then contact the software publisher.

C:\Program Files>mkcert.exe -install
Created a new local CA 🌟
The local CA is now installed in the system trust store! ⚡

C:\Program Files>
```

What does this do?

This installs a **trusted local Certificate Authority (CA)** on your computer. It allows your browser to trust certificates generated by mkcert.

Step 3: Open Your Django Project

Navigate to your Django project folder.

Example:

```
cd "C:\Users\Next webby 01\Desktop\Wisconsin Project\wisconsin"
```

Step 4: Generate HTTPS Certificates

Run:

```
"C:\Program Files\mkcert.exe" 127.0.0.1 localhost 192.168.1.63
```

Replace:

192.168.1.63

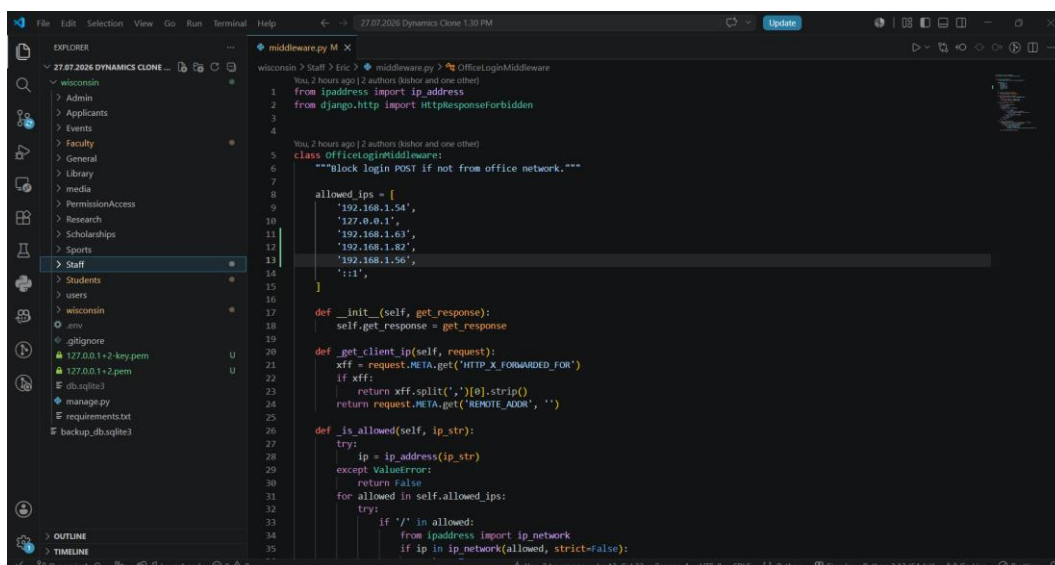
with your own computer's IP address.

After running the command, two certificate files will be created.

Example:

127.0.0.1+2.pem

127.0.0.1+2-key.pem



Step 5: Install Required Packages

Install Django Extensions and Werkzeug.

pip install django-extensions Werkzeug

Step 6: Update Django Settings

Open **settings.py**

Add:

```
INSTALLED_APPS = [
    ...
    "django_extensions",
]
```

Save the file.

Step 7: Start Django with HTTPS

Run:

```
python manage.py runserver_plus 0.0.0.0:8000 --cert-file 127.0.0.1+2.pem --key-file 127.0.0.1+2-key.pem
```

If everything is configured correctly, you should see:

Running on <https://127.0.0.1:8000>

Running on <https://192.168.1.63:8000>

Step 8: Open the Website

On your own computer:

<https://127.0.0.1:8000>

or

<https://192.168.1.63:8000>

On another computer connected to the same network:

<https://192.168.1.63:8000>

Replace the IP address with your own system's IP if it changes.

Step 9: Browser Certificate Warning

On Your Computer

Since you installed the local CA using `mkcert -install`, your browser should trust the certificate.

On Another Computer

The browser may display:

- Not Secure
- Your connection isn't private

This happens because the other computer does not trust your local Certificate Authority.

For internal development and testing, you can continue after accepting the warning.

If you want to remove the warning completely, you must also install the mkcert local CA on that computer.

Step 10: Test Push Notifications

After opening the website over HTTPS:

1. Log in.
2. Allow browser notification permission when prompted.
3. Verify that the Service Worker is registered.
4. Send a test notification.

Your browser should now receive push notifications successfully.

Files Created

During the setup, mkcert creates certificate files similar to:

127.0.0.1+2.pem
127.0.0.1+2-key.pem

These files are required only for local HTTPS development.